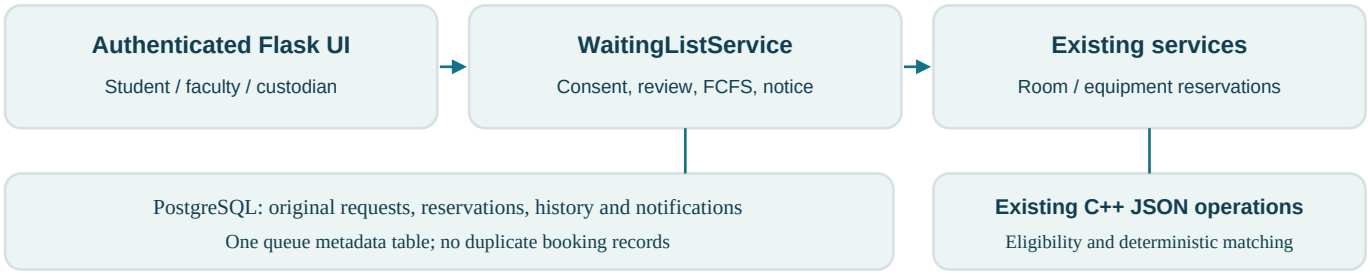
Iteration 47

Resource Waiting Lists and Automatic Assignment

Najib Attar | 21 October 2026 | UC-STU-05 | Flask, PostgreSQL and existing C++ JSON engine

1. Objective and delivered workflow

Extend existing room and equipment requests with one consented waiting-list capability. Students accept policy **waiting-lists-v1**, responsible faculty staff pre-approve or reject, and newly available resources are assigned safely without a second offer acceptance. Pending and pre-approved entries hold nothing.



2. Components, reuse and database design

Component	Responsibility / change
WaitingListService	New shared coordinator: immutable consent, contextual FCFS positions, authority/notice rechecks, cancellation and automatic promotion.
waiting_list_entry / 0047a	One new table linked uniquely to exactly one original room or borrowing request. Owner/faculty consistency, approval constraints, indexed queues and idempotent submission identity.
ApplicationState	Nullable positive waiting_notice_minutes; no invented default. Automatic assignment stays disabled until explicitly configured.
Existing room/equipment services	Reuse original parent records, C++ matching, reservation helpers, calendar/faculty locks and normal cancellation/handover/return lifecycles.
Notifications / routes / Jinja	Reuse event history and private inbox; new policy, queue, detail and configuration views use the shared role-aware application shell.
EngineGateway / C++	Existing room and equipment JSON contracts and all five academic allocation strategies remain unchanged; no SQL dependency in C++.

3. Lifecycle and behavior

Pending Review -> Preapproved -> Assigned. Rejected, Cancelled, Expired and Ineligible entries have no active position. Assigned rooms become Confirmed; equipment becomes Approved/reserved, not physically handed over. Later actions use the original reservation/loan workflow.

There are 48 application tables plus Alembic. The physical/high-level ER diagrams and a focused waiting-list ER diagram reflect the actual schema. Existing Strategy and gateway/service boundaries are reused; no new matching framework or second notification system was introduced.

4. Correctness, privacy and failure handling

Concern	Implemented protection
Concurrent assignment	Room promotions share the calendar-state lock with normal bookings/publication. Equipment promotions use faculty locks, with a shared policy lock acquired first. PostgreSQL overlap/aggregate constraints remain authoritative.
Fairness and positions	Stable join IDs give FCFS among eligible requests. Displayed positions group the same faculty, interval and mandatory requirements; different compatible queues can still compete.
Consent and notifications	Version/time and immutable submission fingerprints are recorded. First top-three entry and reaching one notify once. Optional professor invitees receive assignment updates, not private queue positions.
Notice and expiry	Explicit positive notice is required. Larger recorded minima are never reduced. Insufficient notice pauses promotion; unassigned entries expire at their requested start without invented penalties.
Failures and cancellation	Assignment, reservations and notifications commit together or roll back together. Cancellation/rejection and position updates commit before a retryable refresh, so an engine outage cannot undo a withdrawal.
Equipment integrity	The entire quantity must be feasible. Due dates/return intent do not release held units; verified damaged returns stay unavailable. Physical handover remains separate.

5. Verification evidence

Verification	Actual result
Fresh C++ build and complete suite	172 passed, 0 failed; two isolated standalone CLI runtime smokes passed.
Complete Python/PostgreSQL suite	370 run: 369 passed, 0 failed, 1 skipped. The Windows skip is the Linux-only executable-permission check.
Waiting-list suite (included above)	48 passed, 0 failed. Real engine and isolated PostgreSQL schemas; migration/drift, FCFS, notifications, rollback and competing threads/processes.
Browser acceptance	Actual Chrome/Playwright login, consent, cancellation, pre-approval, rejection, notifications, room/equipment automatic assignment and privacy denial. Desktop 1440px and mobile 390px inspected.
Regression and data safety	Existing appointments/calendar, all five strategies, JSON, CSV CLI/import compatibility and database recovery tested. Operational database and four preexisting generated CSV files preserved.

The scenario expansion reproduced a genuine defect before repair: an engine failure while refreshing other waiters rolled back cancellation. The focused fix moves refresh after the withdrawal/decision commit. Regression tests now protect room and equipment cancellation plus rejection, alongside notification-atomicity and normal-approval/promotion races.

6. Operating limits and review boundary

Queue positions are contextual, not guaranteed global ranks. A periodic operating-system timer may invoke **scripts/process_waiting_lists.py** for expiry/recovery; no durable outbox or background broker is introduced. A crash after release can delay promotion until the next inbox/release/maintenance pass. Sequential faculty passes suit the current single-machine scale.

Back up and apply Alembic head **0047a** before serving the new pages; explicitly configure minimum notice. This report records local verification, not deployment or merge approval. CI and draft-PR status are recorded on the branch. Detailed expected outcomes are in **docs/testing/waiting-list-test-cases.md**.